

Universitatea Tehnică din Cluj-Napoca
Catedra de Calculatoare

Structura sistemelor de calcul
Raport proiect

**Implementarea unui perceptron:
standard vs. optimizat SIMD**

Student: Pruteanu Marius

Grupa: 30235

Data: 21 ianuarie 2026

Cuprins

1	Rezumat	2
2	Introducere	3
3	Fundamentare teoretică	4
4	Proiectare și implementare	5
4.1	Arhitectura generală	5
4.2	Moduri de rulare	5
4.3	Detalii de implementare SIMD	5
5	Rezultate experimentale	6
5.1	Metodologia de măsurare	6
5.2	Rezultate	6
6	Concluzii	7
	Bibliografie	8
	Anexe	9

1 Rezumat

Acest proiect a implementat un perceptron pentru clasificare binară în două variante: o versiune standard (scalară) și o versiune optimizată folosind instrucțiuni SIMD (AVX2 / NEON). S-a urmărit reducerea timpului de antrenare prin vectorizarea operațiilor dominante (produs scalar și actualizarea ponderilor), menținând aceeași logică de învățare. Implementarea a fost realizată în C++ și testată pe seturi de date liniare, iar performanța a fost evaluată prin măsurarea timpului de antrenare și a acurateții. Rezultatele au indicat o îmbunătățire a timpului de execuție pentru varianta SIMD, păstrând comportamentul general al algoritmului.

2 Introducere

Perceptronul este unul dintre cele mai simple modele de învățare automată pentru clasificare binară, având avantajul unei reguli de actualizare ușor de implementat. În același timp, performanța devine importantă atunci când numărul de eșantioane și dimensiunea vectorilor de intrare cresc. Instrucțiunile SIMD (Single Instruction Multiple Data) permit procesarea simultană a mai multor valori în cadrul unei singure instrucțiuni, fiind utile pentru operații vectoriale precum produsul scalar.

Scopul proiectului a fost realizarea a două implementări ale perceptronului: (1) o versiune standard și (2) o versiune optimizată SIMD, cu accent pe vectorizarea operațiilor costisitoare. Obiectivele principale au fost: implementarea corectă a regulii perceptronului, compararea performanțelor între cele două variante și prezentarea rezultatelor experimentale.

În continuare, Secțiunea 3 prezintă fundamentele teoretice, Secțiunea 4 descrie proiectarea și implementarea, Secțiunea 5 prezintă rezultatele experimentale, iar Secțiunea 6 formulează concluziile și direcțiile de dezvoltare.

3 Fundamentare teoretică

Perceptronul calculează o sumă ponderată a intrărilor:

$$s = \sum_{i=1}^n w_i x_i + b$$

iar decizia de clasificare pentru două clase este dată de funcția semn:

$$\hat{y} = \text{sign}(s)$$

Regula de învățare actualizează ponderile atunci când există eroare de clasificare:

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i, \quad b \leftarrow b + \eta(y - \hat{y})$$

unde η este rata de învățare.

Pentru optimizarea performanței, operațiile dominante sunt produsul scalar și actualizarea ponderilor. SIMD permite efectuarea în paralel a mai multor înmulțiri și adunări pentru a accelera calculul produsului scalar. În varianta AVX2, se pot procesa 8 valori de tip `float` simultan.

Noțiunile de arhitectură și optimizare low-level prin vectorizare sunt discutate în literatura clasică de organizare a calculatoarelor [1]. De asemenea, documentația Intel pentru intrinsics SIMD oferă detalii despre seturile de instrucțiuni și bune practici de utilizare [2].

4 Proiectare și implementare

Proiectul a fost realizat ca implementare software în C++ (MSVC / Visual Studio). S-au implementat două clase: **Perceptron** (standard) și **PerceptronSIMD** (optimizat). Arhitectura este modulară: generare date, antrenare, predicție, măsurare performanță.

4.1 Arhitectura generală

Componentele principale sunt:

- Generator de seturi de date (liniar separabil și XOR).
- Perceptron standard: produs scalar + actualizare ponderi scalară.
- Perceptron SIMD: produs scalar + actualizare ponderi vectorizate (AVX2 / NEON).
- Modul de testare: comparație timp și acuratețe.

4.2 Moduri de rulare

Aplicația suportă moduri de funcționare:

- `-mode standard`: rulează varianta scalară.
- `-mode simd`: forțează SIMD (dacă AVX2 este disponibil).
- `-mode auto`: selectează automat SIMD dacă platforma suportă AVX2.

4.3 Detalii de implementare SIMD

În varianta SIMD, produsul scalar a fost optimizat prin încărcarea în registre SIMD și acumularea parțială (AVX2: 8 elemente `float`/iterație; NEON: 4 elemente/iterație), urmată de reducere orizontală. Actualizarea ponderilor folosește multiplicarea vectorială cu un scalar (`update`) și adunarea rezultatului la vectorul de ponderi.

5 Rezultate experimentale

Testele au fost rulate în configurația **Release x64** în Visual Studio, cu activarea AVX2 prin opțiunea `/arch:AVX2`. Seturile de date au inclus probleme liniare separabile cu număr variabil de eșantioane și dimensiuni ale vectorilor de intrare.

5.1 Metodologia de măsurare

Pentru fiecare configurație s-a măsurat timpul total de antrenare pe un număr fix de epoci și s-a calculat acuratețea pe setul de antrenare.

5.2 Rezultate

Tabelul 1 exemplifică formatul de raportare; completează cu valorile obținute din rulările tale.

Tabela 1: Comparație performanță: Standard vs SIMD (exemplu).

Eșantioane	Dimensiune	Timp standard (s)	Timp SIMD (s)	Speedup
500	8	0.012	0.008	1.50x
2000	16	0.090	0.055	1.64x

Interpretarea rezultatelor trebuie să discute unde apare câștigul, ce overhead există (de ex. la dimensiuni mici), și ce limitări sunt (de ex. dependența de AVX2).

6 Concluzii

Proiectul a realizat o implementare funcțională a perceptronului în două variante: standard și optimizată SIMD. Optimizarea a vizat operațiile principale (produs scalar și update) și a condus la reducerea timpilor de execuție în testele comparative, păstrând comportamentul general al algoritmului.

Contribuțiile principale au fost: proiectarea modulară, implementarea AVX2/NEON pentru componentele critice și integrarea modurilor de rulare. Ca direcții viitoare se pot considera: extinderea spre perceptron multi-layer (MLP), citirea seturilor de date din fișiere CSV și optimizări suplimentare (FMA, aliniere memorie).

Bibliografie

- [1] John L. Hennessy and David A. Patterson. *Organizarea și proiectarea calculatoarelor – interfața hardware/software*. Editura All Educational, București, 2002.
- [2] Intel Corporation. Intel intrinsics guide, 2024. Documentație online pentru instrucțiuni SIMD (SSE/AVX/AVX2).

Anexa A

Capturi de ecran / Rulări

```

C:\Users\prute\Desktop\SSC_ x + v
PERCEPTRON WITH SIMD OPTIMIZATIONS
=====
Compiled with AVX2 support: YES
CPU AVX2 available: YES
Mode selected: SIMD (mode=auto)
=====
MAIN MENU
=====
1. Performance Comparison
2. XOR Problem Demonstration S
3. Exit
=====
Enter your choice (1-3): |
  
```

Figura 1: Afișarea informațiilor despre platformă și modul de rulare.

```

C:\Users\prute\Desktop\SSC_ x + v
PERCEPTRON WITH SIMD OPTIMIZATIONS
=====
Compiled with AVX2 support: YES
CPU AVX2 available: YES
Mode selected: SIMD (mode=auto)
=====
MAIN MENU
=====
1. Performance Comparison
2. XOR Problem Demonstration S
3. Exit
=====
Enter your choice (1-3): 1
=====
PERFORMANCE COMPARISON: SCALAR vs SIMD
=====
=====
Samples   Features   Scalar (s)   SIMD (s)     Speedup  Acc S / Acc V
=====
500       8          0.000       0.000       1.164x  97.200% 97.200%
2000      16         0.002       0.001       1.507x  97.900% 97.900%
5000      32         0.007       0.003       2.128x  98.140% 98.140%
10000     64         0.031       0.010       3.107x  98.480% 98.480%
=====
MAIN MENU
  
```

Figura 2: Rezultatele benchmark-ului: comparație Standard vs SIMD.

Anexa B

Fragmente de cod sursă

B.1 Interfața clasei PerceptronSIMD

Listing 1: Structura clasei PerceptronSIMD (din perceptron_simd.h)

```

1  #ifndef PERCEPTRON_SIMD_H
2  #define PERCEPTRON_SIMD_H
3
4  #include "perceptron.h"
5  #include <string>
6
7  #ifdef __ARM_NEON
8  #include <arm_neon.h>
9  #elif defined(__AVX2__)
10 #include <immintrin.h>
11 #endif
12
13 class PerceptronSIMD : public Perceptron {
14 public:
15     PerceptronSIMD(size_t input_dim, float lr = 0.01f);
16     PerceptronSIMD(size_t input_dim, float lr, unsigned int seed);
17
18     void train(const std::vector<std::vector<float>>& inputs,
19               const std::vector<int>& targets,
20               size_t epochs) override;
21
22     int predict(const std::vector<float>& input) const override;
23     std::vector<int> predict_batch(const std::vector<std::vector<
24                                     float>>& inputs) const override;
25
26     double train_and_measure(const std::vector<std::vector<float>>&
27                               inputs,
28                               const std::vector<int>& targets,
29                               size_t epochs) override;
30
31     std::string get_type() const override { return "SIMD"; }
32
33     void reset_weights() override;
34 private:

```

```

34     float dot_product(const std::vector<float>& input) const
        override;
35     void weight_update(const std::vector<float>& input, int target,
        int prediction) override;
36
37     float simd_dot_product_optimized(const std::vector<float>&
        input) const;
38     void simd_weight_update_optimized(const std::vector<float>&
        input, int target, int prediction);
39 };
40
41 #endif

```

B.2 Detectarea suportului AVX2 și selectarea modului de rulare

Listing 2: Detectie AVX2 + parse -mode (din main.cpp)

```

1  #if defined(_MSC_VER) && (defined(_M_X64) || defined(_M_IX86))
2  #include <intrin.h>
3  #endif
4
5  static bool cpu_supports_avx2() {
6  #if defined(_MSC_VER) && (defined(_M_X64) || defined(_M_IX86))
7      int info[4] = { 0,0,0,0 };
8
9      __cpuid(info, 1);
10     bool osxsave = (info[2] & (1 << 27)) != 0;
11     bool avx = (info[2] & (1 << 28)) != 0;
12     if (!osxsave || !avx) return false;
13
14     unsigned long long xcr0 = _xgetbv(0);
15     bool xmm_ok = (xcr0 & 0x2) != 0;
16     bool ymm_ok = (xcr0 & 0x4) != 0;
17     if (!xmm_ok || !ymm_ok) return false;
18
19     __cpuidex(info, 7, 0);
20     bool avx2 = (info[1] & (1 << 5)) != 0; // EBX bit 5
21     return avx2;
22 #else
23     return false;
24 #endif
25 }

```

```

26
27 static std::string get_mode(int argc, char** argv) {
28     std::string mode = "auto";
29
30     for (int i = 1; i < argc; ++i) {
31         std::string a = argv[i];
32
33         if (a == "--mode" && i + 1 < argc) {
34             mode = argv[i + 1];
35             break;
36         }
37         const std::string pfx = "--mode=";
38         if (a.rfind(pfx, 0) == 0) {
39             mode = a.substr(pfx.size());
40             break;
41         }
42     }
43
44     std::transform(mode.begin(), mode.end(), mode.begin(),
45         [](unsigned char c) { return (char)std::tolower(c); });
46
47     if (mode != "auto" && mode != "simd" && mode != "standard")
48         mode = "auto";
49     return mode;
50 }
51
52 static bool resolve_use_simd(const std::string& mode) {
53     bool avx2_ok = cpu_supports_avx2();
54
55     if (mode == "standard") return false;
56     if (mode == "simd") return avx2_ok;
57     return avx2_ok; // auto
58 }
59
60 static std::unique_ptr<Perceptron> make_model(size_t features,
61     float lr, unsigned int seed, bool use_simd) {
62     if (use_simd) return std::make_unique<PerceptronSIMD>(features,
63         lr, seed);
64     return std::make_unique<Perceptron>(features, lr, seed);
65 }

```

B.3 Produs scalar optimizat SIMD (AVX2 / NEON)

Listing 3: Dot product SIMD (din perceptron_simd.cpp)

```

1 float PerceptronSIMD::dot_product(const std::vector<float>& input)
  const {
2     return simd_dot_product_optimized(input);
3 }
4
5 float PerceptronSIMD::simd_dot_product_optimized(const std::vector<
  float>& input) const {
6     const float* w = weights.data();
7     const float* x = input.data();
8     float result = 0.0f;
9
10    size_t i = 0;
11    size_t n = input_size;
12
13    #ifdef __AVX2__
14        __m256 sum = _mm256_setzero_ps();
15        for (; i + 7 < n; i += 8) {
16            __m256 vw = _mm256_loadu_ps(w + i);
17            __m256 vx = _mm256_loadu_ps(x + i);
18            sum = _mm256_add_ps(sum, _mm256_mul_ps(vw, vx));
19        }
20        __m128 s128 = _mm_add_ps(_mm256_extractf128_ps(sum, 0),
21            _mm256_extractf128_ps(sum, 1));
22        s128 = _mm_hadd_ps(s128, s128);
23        s128 = _mm_hadd_ps(s128, s128);
24        result = _mm_cvtss_f32(s128);
25
26    #elif defined(__ARM_NEON)
27        float32x4_t sum = vdupq_n_f32(0.0f);
28        for (; i + 3 < n; i += 4) {
29            float32x4_t vw = vld1q_f32(w + i);
30            float32x4_t vx = vld1q_f32(x + i);
31            sum = vaddq_f32(sum, vmulq_f32(vw, vx));
32        }
33        float32x2_t s2 = vadd_f32(vget_high_f32(sum), vget_low_f32(sum)
34            );
35        result = vget_lane_f32(vpadd_f32(s2, s2), 0);
36    #endif

```

```

36
37     for (; i < n; ++i) result += w[i] * x[i];
38     return result;
39 }

```

B.4 Actualizarea ponderilor optimizată SIMD (AVX2 / NEON)

Listing 4: Weight update SIMD (din perceptron_simd.cpp)

```

1 void PerceptronSIMD::weight_update(const std::vector<float>& input,
2   int target, int prediction) {
3     simd_weight_update_optimized(input, target, prediction);
4 }
5
6 void PerceptronSIMD::simd_weight_update_optimized(const std::vector
7   <float>& input, int target, int prediction) {
8     float error = static_cast<float>(target - prediction);
9     float update = learning_rate * error;
10
11     float* w = weights.data();
12     const float* x = input.data();
13     size_t i = 0;
14
15 #ifdef __AVX2__
16     __m256 u = _mm256_set1_ps(update);
17     for (; i + 7 < input_size; i += 8) {
18         __m256 vx = _mm256_loadu_ps(x + i);
19         __m256 vw = _mm256_loadu_ps(w + i);
20         vw = _mm256_add_ps(vw, _mm256_mul_ps(u, vx));
21         _mm256_storeu_ps(w + i, vw);
22     }
23
24 #elif defined(__ARM_NEON)
25     float32x4_t u = vdupq_n_f32(update);
26     for (; i + 3 < input_size; i += 4) {
27         float32x4_t vx = vld1q_f32(x + i);
28         float32x4_t vw = vld1q_f32(w + i);
29         vw = vaddq_f32(vw, vmulq_f32(u, vx));
30         vst1q_f32(w + i, vw);
31     }
32 #endif

```

```

32     for (; i < input_size; ++i) w[i] += update * x[i];
33     bias += update;
34 }

```

B.5 Referință: variantă standard (scalară) pentru comparație

Listing 5: Dot product + update scalar (din perceptron.cpp)

```

1     std::uniform_real_distribution<float> dist(-scale, scale);
2
3     weights.resize(input_size);
4     for (size_t i = 0; i < input_size; ++i) weights[i] = dist(gen);
5     bias = dist(gen);
6 }
7
8 void Perceptron::reset_weights() {
9     initialize_weights();
10 }
11
12 float Perceptron::activation(float x) const {
13     return x >= 0 ? 1.0f : -1.0f;
14 }
15
16 float Perceptron::dot_product(const std::vector<float>& input)
17     const {
18     float result = 0.0f;
19     for (size_t i = 0; i < input_size; ++i) {
20         result += weights[i] * input[i];
21     }
22     return result;
23 }
24
25 int Perceptron::predict(const std::vector<float>& input) const {
26     if (input.size() != input_size) {
27         throw std::invalid_argument("Input size doesn't match
28             perceptron input size");
29     }
30     float weighted_sum = dot_product(input) + bias;
31     return activation(weighted_sum) >= 0 ? 1 : -1;
32 }
33
34 }

```



```
32 std::vector<int> Perceptron::predict_batch(const std::vector<std::
    vector<float>>& inputs) const {
33     std::vector<int> preds;
34     preds.reserve(inputs.size());
35     for (const auto& in : inputs) preds.push_back(predict(in));
36     return preds;
37 }
38
39 void Perceptron::weight_update(const std::vector<float>& input, int
    target, int prediction) {
40     float error = static_cast<float>(target - prediction);
41     float update = learning_rate * error;
42
43     for (size_t i = 0; i < input_size; ++i) {
44         weights[i] += update * input[i];
45     }
46     bias += update;
47 }
48
49 void Perceptron::train(const std::vector<std::vector<float>>&
    inputs,
50     const std::vector<int>& targets,
51     size_t epochs) {
52
53     if (inputs.size() != targets.size()) {
54         throw std::invalid_argument("Inputs and targets must have
            the same size");
55     }
56
57     for (size_t epoch = 0; epoch < epochs; ++epoch) {
58         int correct = 0;
```